

Arquitectura del Sistema:

Flujo de Datos (Pipeline)

El sistema se ejecuta en bucle (por ejemplo, cada 4 horas)

1. FASE DE INGESTA:

Recolecta la información cruda.

Inputs:

Precios: Velas OHLCV de los x activos

Noticias: Últimos 20 titulares de cada activo.

Macro: S&P500, VIX, Índice Fear & Greed.

Procesamiento: Calcular indicadores técnicos y limpiar el texto de las noticias si hay cosas innecesarias en cuanto a formato.

2. FASE CUANTITATIVA (ML para una buena base)

Objetivo:

Establecer una base matemática sólida utilizando ML tradicional. Detectar ineficiencias estadísticas en el mercado mediante un modelo de regresión no lineal. El sistema no busca predecir el precio, sino el rendimiento relativo de un activo frente a sus pares.

Input(X):

Para que el modelo entienda el estado actual de la tendencia sin romper las reglas estadísticas, utilizamos técnicas avanzadas de transformación de series temporales.

Diferenciación Fraccional $d \approx 0.4$: A diferencia de los retornos simples (borran memoria) o los precios brutos (no estacionarios), aplicamos diferenciación fraccional. Esto permite al modelo "ver" la memoria de largo plazo manteniendo la serie matemáticamente estacionaria.

Indicadores Normalizados: RSI, Volatilidad Relativa y Posición en Bandas de Bollinger entre otros.

Output o Target(Y):

Entrenamos al modelo para predecir el Retorno Excedente a corto plazo (x ej. 4 horas).

La lógica que seguimos es que el modelo aprende la función de mapeo Contexto $\rightarrow$ Rentabilidad (x.ej. Cuando el precio diferenciado está en extremos históricos (Contexto) y el RSI diverge, el Alpha futuro tiende a ser negativo (Reversión a la media))

Modelo: XGBoost Regressor (Predicción de Retorno Relativo).

En lugar de intentar predecir el precio absoluto futuro (una serie temporal no estacionaria y ruidosa), utilizamos un enfoque de Cross-Sectional Regression.

Cross-Sectional Regression:

En lugar de mirar una sola moneda en el tiempo, el modelo evalúa todos los activos simultáneamente en cada corte temporal.

Básicamente, el modelo no predice cuánto valdrá Bitcoin, sino cuánto se desviará Bitcoin del promedio del mercado.

Lógica del Alpha (Ranking Relativo): Nuestra necesidad para la optimización de carteras es maximizar la exposición a activos con mayor probabilidad de rendimiento superior.

Transformamos el problema de "predicción de precios" a "predicción de Alpha".

Enfoque Tradicional (el malo): Tratar de adivinar si BTC sube un 3.5%. Si el mercado global cae por un evento macro, el modelo falla estrepitosamente.

Nuestro Enfoque: Predecir el diferencial respecto a la media. Ejemplo: Si mañana el mercado colapsa un 10% (promedio), pero nuestro modelo predijo que BTC es fuerte, quizás BTC solo caiga un 8%.

Matemática: $R_{\{btc\}} (-8\%) - R_{\{promedio\}} (-10\%) = \mathbf{+2\%}$.

Este +2% es nuestro Alpha. Aunque perdamos valor nominal, hemos ganado en términos relativos, protegiendo la cartera mejor que el índice. Esto hace que el modelo sea robusto y estacionario, capaz de aprender patrones universales que funcionan tanto en euforia como en pánico:

Lo podemos ver claramente que tanto en un bull como bearish run el modelo seguirá aprendiendo y no se confundirá, encontrando patrones relevantes:

Bull Run:

- Moneda A: +20%
- Moneda B: +15%
- Promedio: +17.5%
- Ranking: $A > B$.

Crash:

- Moneda A: -5%
- Moneda B: -10%
- Promedio: -7.5%
- Ranking: $A > B$.

Salidas:

El modelo genera dos salidas críticas para las siguientes fases:

- Ranking Ordenado: Lista de prioridad (1. SOL, 2. ETH... 8. DOGE) para pasarlo al LLM
- Vector de Views: Un vector numérico con la magnitud exacta del retorno relativo esperado (x ej: SOL: +0.02, BTC: +0.005, DOGE: -0.01). Este dato numérico es para el modelo de Black-Litterman en la ultima fase.

3. FASE AGENTES (MoE) - mezcla de cada agente como experto

Objetivo: Introducir comprensión contextual al sistema. Mientras el ML ve números, los Agentes LLM ven riesgos fundamentales que no aparecen en las gráficas hasta que es tarde.

Nivel 1

Convertimos al LLM en un Auditor de Señales. No le pedimos que calcule precios (es malo porque es un modelo generativo), le pedimos que valide la tesis del modelo matemático.

Input:

Rankind del modelo ML: "El modelo cuantitativo indica que SOL es la oportunidad #1 (Bullish)"

Noticias desde API o donde sea sobre ese activo:

Prompt ejemplo:

"Actúa como un Analista Senior de Riesgos. Tu modelo cuantitativo ha emitido una señal de compra fuerte para [ACTIVO]. Revisa las noticias recientes: ¿Existe algún evento fundamental que invalide esta señal numérica?"

Output:

Conviction Score (0.0 a 1.0)

Nivel 2

Este agente no mira activos individuales, sino el contexto global para calibrar la agresividad de la cartera.

Input: Datos Macro (VIX, S&P500, Tasas de Interés, Índice Fear & Greed).

Prompt del Sistema: Analiza el entorno macroeconómico. ¿Estamos en un régimen de "Risk-On" (apetito de riesgo) o "Risk-Off" (protección de capital)?

Output Crítico: Risk Aversion Parameter (δ): Un valor numérico que ajusta la fórmula de optimización. Miedo extremo $\rightarrow$ Delta Alto (El modelo matemático priorizará bonos/USDT). Euforia/Estabilidad $\rightarrow$ Delta Bajo (El modelo permitirá más exposición a volatilidad).

3. FASE DE OPTIMIZACIÓN

Eliminar la alucinación financiera, queremos algo matemáticamente consistente y preciso, utilizamos un motor de optimización convexa determinista: Black-Litterman.

Qué recibe:

Vector de Retornos Relativos generados por el ML con el conviction que nos devolvió el LLM y la aversión de riesgo del LLM que sigue la estrategia de los datos Macro, y por último las restricciones que queramos.

EL output final es un JSON con la asignación de pesos matemáticamente óptima (x.ej: {'BTC': 0.221, 'ETH': 0.184, ...}).